

이슈 접수 채널 자동화

■ 배경 및 문제 인식

- 사업부와 개발팀 간 이슈 소통 방식의 구조적 문제 인식
- 이슈 접수 채널이 네이버웍스, 이메일, 슬랙 등 여러 곳으로 분산
- 이슈 누락 및 중복 접수 발생 가능성 상존
- 처리 담당자 및 진행 상태 파악 어려움
- 사업부가 개발팀 내부 담당자 배정까지 신경써야 하는 비효율 발생
- 업무 히스토리가 메신저에만 남아 담당자 부재 시 업무 연속성 단절

문제	영향
이슈 접수 채널 분산	누락 및 중복 접수 발생
처리 이력이 메신저에만 존재	담당자 부재 시 업무 연속성 단절
담당자 배정 기준 불명확	특정 인원에게 업무 쏠림 현상
사업부가 처리 진행 상태 파악 불가	반복 문의 및 커뮤니케이션 비용 증가

■ 개선 방향

문제 해결을 위해 3가지 원칙을 설정

채널 일원화

- 개발팀 이슈 접수 채널을 슬랙으로 단일화
- 네이버웍스, 이메일 등 기존 채널로 유입되는 이슈는 보고자가 슬랙에 직접 입력
- 슬랙에 입력된 이슈는 자동으로 Jira로 연동

이력 자동 저장

- 슬랙에 등록된 이슈는 자동으로 Jira 티켓으로 전환되어 처리 이력 보존
- 이후 모든 소통은 슬랙 스레드와 Jira 댓글이 양방향으로 동기화되어 두 곳 모두 기록

담당자 자동 배정

- 사업부는 개발팀 내부 담당자를 별도로 고민하지 않아도 됨
- 이슈 접수 시점에 개발팀 내부 프로세스에 의해 담당자 자동 부여
- 이슈 세부 진행 사항은 Jira를 통해 실시간 확인 가능

■ 개선된 점

항목	As-Is	To-Be
이슈 접수 채널	슬랙, 이메일, 네이버웍스 등 분산	슬랙 단일 채널로 통합
이슈 처리 이력	메신저 대화에 소실	Jira에 자동 기록
담당자 배정	수동 지정 또는 특정인 쏠림	자동 배정
처리 상태 확인	사업부가 개별 문의 필요	슬랙 스레드 및 Jira에서 실시간 확인
소통 이력	슬랙 또는 Jira 한쪽에만 존재	양쪽 모두 자동 동기화

■ 현재 한계 및 후속 과제

현재 한계

- 자동화 적용 범위는 이슈 생성, 담당자 배정, 댓글 동기화 수준에 한정
- 슬랙을 통해 접수된 이슈에 한해서만 자동화 적용

후속 과제

- 이슈 유형별 분류 및 담당자 분배 배정 고도화
- 개발팀으로 이관된 이슈에 대한 개발팀에서 고객 답변 프로세스 개선 방안 검토
 - 답변 정규화, AI 활용 등 방안 고민 필요

■ 시스템 구성

사용 기술 스택

역할	도구
이슈 접수 및 소통	Slack
이슈 트래킹	Jira
자동화 오케 스트레이션	N8N
담당자 및 이 력 관리	PostgreSQL

■ 자동화 구축

STEP 1. 준비물 확인

계정 및 접속 정보

항목	확인 방법
슬랙 워크스페이스 관리자 권한	워크스페이스 설정에서 역할 확인
Jira 프로젝트 관리자 권한	프로젝트 설정 접근 가능 여부 확인
N8N 접속 URL 및 계정	팀 N8N 서버 주소 확인
PostgreSQL 접속 정보	DB 호스트/포트/계정/비밀번호 확인

체크리스트

```
1 [ ] 슬랙 Bot Token: xoxb-10898682925922-10892336139683-0aH0FtKU1Sx5I...
2 [ ] 슬랙 Signing Secret: 45cd0e00cd9d4a98b20f9...
3 [ ] 슬랙 채널 ID: C0AS8JW4PUK
4 [ ] Jira 도메인: https://well-check.atlassian.net
5 [ ] Jira 이메일: esseo@idstrust.com
```

```
6 [ ] Jira API Token: ATATT3xFfGF0iePqsV1R1o70BtWDw6UThng4qJS7vBrpWjuYC...
7 [ ] Jira 프로젝트 키: MCCWC
8 [ ] Jira slack_channel_id 커스텀 필드 ID: customfield_10407
9 [ ] Jira slack_message_ts 커스텀 필드 ID: customfield_10408
10 [ ] Jira slack_thread_ts 커스텀 필드 ID: customfield_10409
11 [ ] N8N 웹훅 URL (Slack 이벤트):
12 [ ] N8N 웹훅 URL (Jira 이벤트):
```

STEP 2. 슬랙 앱 생성 및 설정

2-1. 슬랙 앱 생성

- <https://api.slack.com/apps> 접속
- 우측 상단 **Create New App** 클릭
- **From scratch** 선택
- App Name에 (예: `wellcheck-bot`) 입력
- 워크스페이스 선택 후 **Create App** 클릭

2-2. Bot 활성화

앱만 만들면 봇이 자동으로 생기지 않음

- 생성한 **APP** 클릭
- 좌측 메뉴 **App Home** 클릭
- **Your App's Presence in Slack** 섹션에서 **Edit** 클릭
- Bot Name 입력 (예: `wellcheck-bot`) → **Add** 클릭
- **Always Show My Bot as Online** 토글 ON

2-3. 권한(Scope) 추가

봇이 슬랙에서 할 수 있는 행동을 지정합니다.

1. 좌측 메뉴 **OAuth & Permissions** 클릭
2. 스크롤 내려서 **Scopes** 섹션 → **Bot Token Scopes** 아래 **Add an OAuth Scope** 클릭
3. 아래 권한을 하나씩 검색해서 추가

1	<code>channels:history</code>	메시지 읽기
2	<code>channels:read</code>	채널 정보 조회
3	<code>chat:write</code>	메시지 전송
4	<code>reactions:read</code>	이모지 감지
5	<code>users:read</code>	유저 정보 조회

4. 페이지 최상단으로 스크롤 → **Install to Workspace** 클릭 → 허용
5. **Bot User OAuth Token** 복사 → 메모장 저장 (xoxb-로 시작, 외부 노출 금지)

2-4. Signing Secret 확인

1. 좌측 메뉴 **Basic Information** 클릭
2. **App Credentials** 섹션에서 **Signing Secret** 옆 **Show** 클릭
3. 복사 → 메모장에 저장

2-5. 운영 채널 생성 및 봇 초대

1. 슬랙 워크스페이스에서 새 채널 생성: (예: #이슈접수)
2. 채널명 클릭 → 채널 세부정보 → 맨 아래 채널 ID 복사 → 메모장에 저장
3. 채널에서 아래 입력해서 봇 초대

```
1 | /invite @welcheck-bot
```

STEP 3. Jira 설정

3-1. API Token 발급

1. <https://id.atlassian.com/manage-profile/security/api-tokens> 접속
2. **Create API token** 클릭
3. 이름 입력 (예: welcheck-n8n) → **Create**
4. 토큰 복사 → 메모장에 저장

3-2. 커스텀 필드 추가

슬랙과 Jira를 연결하는 핵심 데이터(슬랙 채널 ID, 메시지 TS)를 Jira 티켓에 저장하기 위해 커스텀 필드를 추가합니다.

1. Jira 상단 메뉴 **설정(톱니바퀴)** → **이슈** 클릭
2. 좌측 메뉴 **필드** → **사용자 정의 필드** → **사용자 정의 필드 만들기** 클릭
3. **텍스트 필드(단일 행)** 선택 → 아래 3개 필드 생성

```
1 | 필드명: slack_channel_id
2 | 필드명: slack_message_ts
3 | 필드명: slack_thread_ts
```

4. 생성 후 각 필드를 프로젝트 이슈 화면에 연결 (필드 설정 → 화면 연결)

3-3. 커스텀 필드 ID 확인

커스텀 필드는 내부적으로 `customfield_XXXXX` 형태의 ID를 가집니다. N8N에서 이 ID가 필요합니다.

브라우저에서 아래 URL 접속 :

```
1 | https://well-check.atlassian.net/rest/api/3/field
```

3-4. 담당자 Account ID 확인

팀원들의 Jira Account ID를 확인합니다.

```
1 | https://well-check.atlassian.net/rest/api/3/users/search?maxResults=200&startAt=0&query=.
```

결과 JSON에서 "accountId" 값을 각 팀원별로 복사 → 메모장에 저장

	이름	accountId
1	서은상	62a2dec5d442e60068433460
2	정종술	5e38c3bd8e3b9c0cbe4ef769
3	박종훈	712020:f6d91eef-594a-4377-b2aa-78ec6af3eeac
4	임수민	712020:9ad7027f-0f25-45de-bba2-5954892d8e6e
5	홍연하	63d86110790148a1809301ae
6	김혜린	63d8607cc2b1cb6b346f7119
7	이현호	636070b976b91b62562f0236
8	정유미	712020:eea335a8-b315-4156-91ae-7627b0ea054d
9	설재혁	6350aa7813f37118d724ef6e
10	이진우	5f9faecb632c62007175c15c
11	강은비	712020:544f9e3d-e4d3-420c-984f-dcd0e8a67eed
12	김영룡	615eb768071a7200715e8448
13	전재연	712020:d8acf339-671f-489b-bc3e-009291062e5d
14	오혁근	712020:540486e2-3153-427d-88bf-35e214f276df
15	이지은	712020:0b16dbd3-b47a-4c16-8ffd-35c3a760246d
16	프로덕트 웰다	712020:d1ce5275-6c22-4cf4-9315-76b46ba32016
17	윤수빈	712020:a704adbf-9e33-4522-9712-9029a0b505fb
18	황원영	712020:7b264e33-9a04-4304-9b48-343d67655afe

1 9	Wellcheck-운영	712020:50c4ff4d-8b7e-4238-b0ef-9c0f4a01dec1
--------	--------------	---

3-5. Slack 담당자 ID 확인

Postman으로 호출

- GET 선택
- URL: `https://slack.com/api/users.list`
- Headers
 - Key: `Authorization`
 - Value: `Bearer xoxb-전체토큰`

	이름	Id
1	서은상	U0AQKE1Q3GF
2	정종술	U0ARDQCBKQR
3	박종훈	U0AQXFATCAZ
4	임수민	U0AQXF7D2P7
5	홍연하	U0AR4FB8C0L
6	김혜린	U0AQYRGCN78
7	이현호	U0AQKE2KFC7
8	정유미	U0AR4FA2WAY
9	설재혁	U0ARDQC42BT
1 0	이진우	U0AQYRCU9BQ
1 1	강은비	U0AQXFANT8V
1 2	장민아	U0AQKE29UUX
1 3	김영룡	U0AQXEM416Z
1 4	전재연	U0AQYRC8T5L
1 5	오혁근	U0AR0T87P18

1 6	김수민	U0AQKE0697Z
1 7	윤수빈	U0AR9N253N3
1 8	김경미	U0ARDQ8PRRP
1 9	황원영	U0ARRLZFG1G

STEP 4. PostgreSQL 테이블 생성

4-1. 테이블 생성

```

1  -- 테이블 삭제 (참조 순서 역순으로 삭제)
2  DROP TABLE IF EXISTS comment_sync_log CASCADE;
3  DROP TABLE IF EXISTS issue_links CASCADE;
4  DROP TABLE IF EXISTS assignees CASCADE;
5
6  -- 담당자 관리 테이블
7  CREATE TABLE assignees (
8      id SERIAL PRIMARY KEY,
9      name VARCHAR(100) NOT NULL,
10     jira_account_id VARCHAR(100) NOT NULL,
11     slack_user_id VARCHAR(100),
12     is_active BOOLEAN DEFAULT true,
13     is_primary BOOLEAN DEFAULT false,
14     created_at TIMESTAMP DEFAULT NOW()
15 );
16
17 -- 슬랙 ↔ Jira 연결 고리 테이블
18 CREATE TABLE issue_links (
19     id SERIAL PRIMARY KEY,
20     slack_channel_id VARCHAR(100) NOT NULL,
21     slack_message_ts VARCHAR(100) NOT NULL UNIQUE,
22     jira_issue_key VARCHAR(50) NOT NULL,
23     assignee_id INT REFERENCES assignees(id),
24     created_at TIMESTAMP DEFAULT NOW()
25 );
26
27 -- 댓글 동기화 중복 방지 테이블
28 CREATE TABLE comment_sync_log (
29     id SERIAL PRIMARY KEY,
30     issue_link_id INT REFERENCES issue_links(id),
31     source VARCHAR(10) NOT NULL,
32     source_comment_id VARCHAR(200) NOT NULL,
33     synced_at TIMESTAMP DEFAULT NOW(),
34     UNIQUE(source, source_comment_id)
35 );

```

4-2. 담당자 데이터 입력

STEP 3-4에서 확인한 Account ID와 STEP 2에서 확인한 슬랙 User ID를 사용합니다.

```

1  INSERT INTO assignees (name, jira_account_id, slack_user_id, rotation_order)
2  VALUES
3      ('홍길동', '여기에_jira_account_id', 'U여기에_slack_user_id', 1),
4      ('김철수', '여기에_jira_account_id', 'U여기에_slack_user_id', 2),

```



```
5 ('이영희', '여기에_jira_account_id', 'U여기에_slack_user_id', 3);
```

4-3. 테이블 생성 확인

```
1 SELECT * FROM assignees;  
2 SELECT * FROM issue_links;  
3 SELECT * FROM comment_sync_log;
```

STEP 5. N8N Credentials 등록

N8N에 접속 후 좌측 메뉴 **Credentials** → **Add Credential** 순서로 아래 3개를 등록합니다.

5-1. 슬랙 Credential

1. Slack API 검색 → 선택
2. **Name:** Slack Bot Token
3. **Access Token:** 메모장의 xoxb- 토큰 붙여넣기
4. **Save**

5-2. Jira Credential

1. Basic Auth 검색 → 선택
2. **Name:** Jira API
3. **Username:** Jira 로그인 이메일
4. **Password:** 메모장의 Jira API Token 붙여넣기
5. **Save**

5-3. PostgreSQL Credential

1. Postgres 검색 → 선택
2. **Name:** Welcheck DB
3. Host, Port, Database, User, Password 입력
4. **Test Connection** 클릭해서 연결 확인 → **Save**

STEP 6. N8N 워크플로우 구성

- [n8n.io - Workflow Automation](https://n8n.io)

STEP 8. Jira Automation 설정

Jira에서 특정 이벤트가 발생하면 N8N 웹훅을 호출하도록 설정합니다.

자동화 규칙 만들기 공통 방법

프로젝트 → 프로젝트 설정 → 자동화 → 규칙 만들기

8-1. 댓글 추가 시 N8N 호출

항목	설정 값
트리거	댓글이 추가됨
조건	slack_message_ts 필드가 비어있지 않음
액션	웹 요청 전송
URL	https://n8n.wellcho.co/webhook/jira-comment
방법	POST
본문	아래 JSON

json

```
1 {
2   "event": "jira_comment_added",
3   "issue_key": "{{issue.key}}",
4   "comment_body": "{{comment.body.rawContent}}",
5   "comment_author": "{{comment.author.displayName}}",
6   "comment_id": "{{comment.id}}",
7   "slack_channel_id": "{{issue.fields.slack_channel_id}}",
8   "slack_message_ts": "{{issue.fields.slack_message_ts}}"
9 }
```

참고사항

무한루프 방지 설계

- 양방향 동기화에서 가장 주의할 점은 무한루프 방지
- 동기화된 댓글에는 각각 [Slack], [Jira] 를 붙여서 재 동기화를 차단
- DB에 동기화 이력을 comment_sync_log 테이블에 기록하고 동일한 댓글은 처리하지 않음